Assignment No.6

Title : Text Classification Using Naive Bayes or SVM

Aim

To design and implement a basic text classification solution in Python using Naive Bayes or Support Vector Machine (SVM) for sentiment or topic classification on a small dataset.

Objectives

1. To preprocess and prepare a text dataset for classification.
2. To implement a text classification model using Naive Bayes or SVM in Python.
3. To evaluate the performance of the classifier using accuracy and other metrics.
4. To analyze and interpret classification results on sample text inputs.

Expected Outcomes

1. Successful training of a text classification model on a small labeled dataset.
2. Ability to classify new/unseen text into predefined categories (e.g., positive/negative or topics).
3. Evaluation of model performance using metrics such as accuracy, precision, recall, and F1-score.
4. Understanding of how Naive Bayes and SVM models work for NLP classification tasks.
5. Improved understanding of feature extraction techniques like TF-IDF and Count Vectorizer.

Theory

1. Introduction to Text Classification

Text classification is a fundamental task in Natural Language Processing (NLP) that involves assigning predefined labels or categories to text data. It is widely used in applications such as spam detection, sentiment analysis, topic labeling, and language identification.

For example:

A movie review can be classified as **Positive** or **Negative** (Sentiment Classification).

A news article can be labeled as **Sports**, **Politics**, or **Technology** (Topic Classification).

Text classification is widely used in:

- Spam detection
- Customer review analysis
- News categorization
- Medical diagnosis from clinical notes
- Chatbot intent recognition

2. Text Preprocessing

Before training any classifier, raw text must be preprocessed to convert it into a machine-readable format. Common preprocessing steps include:

1. Lowercasing – Convert all text to lowercase.

2. Removing Punctuation and Special Characters – Strip unnecessary symbols.
3. Tokenization – Split text into individual words (tokens).
4. Stopword Removal – Remove common words like "the", "is", "and" that carry little meaning.
5. Stemming/Lemmatization – Reduce words to their root forms (e.g., "running" → "run").

3. Feature Extraction

Machine learning models cannot work directly with raw text. Text must be converted into numerical features.

1. Count Vectorizer (Bag of Words)

Converts text into a matrix of word counts.

Formula:

Each document is represented as a vector of word frequencies.

$d = [c1, c2,..., cV]$

Where ci = Count of word wi in the document

2. TF-IDF Vectorizer

Assigns higher weights to important words and lower weights to common words.

Step 1: Term Frequency (TF)

$TF(t, d)$ = Number of times term t appears in d / Total terms in d

Step 2: Inverse Document Frequency (IDF)

$IDF(t) = \log(N / df(t))$

Where:

- N = Total number of documents
- $df(t)$ = Number of documents containing term t

Final TF-IDF Formula

$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$

4. Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' Theorem with an assumption of conditional independence between features (words).

Bayes' Theorem:

$P(C \mid X) = P(X \mid C) \times P(C) / P(X)$

Where:

- $P(C \mid X)$ = Posterior probability of class C given input X
- $P(X \mid C)$ = Likelihood of X given class C
- $P(C)$ = Prior probability of class C
- $P(X)$ = Probability of input X (normalizing constant)

For text classification with words w1, w2,..., wn:

$P(C \mid w1, w2,..., wn) \propto P(C) \times \prod P(wi \mid C)$

Types of Naive Bayes for NLP:

1. Multinomial Naive Bayes – Best suited for word count features (used with Bag of Words/TF-IDF).
2. Bernoulli Naive Bayes – Used when features are binary (word present or absent).
3. Gaussian Naive Bayes – Used for continuous numeric features.

5. Support Vector Machine (SVM) Classifier

Support Vector Machine (SVM) is a supervised machine learning algorithm that finds the optimal hyperplane to separate different classes in the feature space.

Key Concepts:

- Hyperplane – A decision boundary separating two classes.
- Support Vectors – Data points closest to the hyperplane, which define the margin.
- Margin – The distance between the hyperplane and the nearest support vectors. SVM maximizes this margin.

SVM Objective:

Maximize: $2 / \|w\|$

Subject to: $y_i(w \cdot x_i + b) \geq 1$

Where:

- w = Weight vector (normal to the hyperplane)
- b = Bias term
- y_i = Class label (+1 or -1)
- x_i = Feature vector

Kernel Functions for Non-Linear Data:

1. Linear Kernel – Used for linearly separable text data.
2. Polynomial Kernel – Maps data to higher dimensions.
3. RBF Kernel (Radial Basis Function) – Most commonly used for non-linear classification.

6. Evaluation Metrics

After training the classifier, its performance is evaluated using:

1. Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)

2. Precision

Precision = TP / (TP + FP)

3. Recall (Sensitivity)

Recall = TP / (TP + FN)

4. F1-Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)

Where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

7. Naive Bayes vs. SVM – Comparison

Naive Bayes is fast, simple, and works well with small datasets and high-dimensional text features. It is probabilistic and assumes feature independence. SVM is more powerful and accurate, especially with high-dimensional sparse data like TF-IDF vectors. It finds the optimal hyperplane but is computationally more expensive. For text classification tasks, both algorithms work well; however, SVM with a linear kernel often outperforms Naive Bayes on larger datasets, while Naive Bayes remains preferred for real-time and resource-constrained applications.

Pseudocode

```
START
INPUT:
  Text dataset (labeled sentences/documents)
  Labels (categories: Positive/Negative or Sports/Politics/Technology)
  Classifier choice (Naive Bayes or SVM)

STEP 1: Load Dataset
  1. Read text samples and their corresponding labels
  2. Split data into training set and testing set

STEP 2: Preprocess Text
  1. Convert text to lowercase
  2. Remove punctuation and special characters
  3. Tokenize text into words
  4. Remove stopwords
  5. Apply stemming or lemmatization (optional)

STEP 3: Feature Extraction
  1. Apply TF-IDF Vectorizer or Count Vectorizer
  2. Transform training and testing texts into feature vectors

STEP 4: Train Classifier
  IF Naive Bayes:
     Train MultinomialNB model on training feature vectors and labels
```

IF SVM:
Train LinearSVC or SVC model on training feature vectors and labels

STEP 5: Test Classifier
1. Pass test feature vectors to trained model
2. Predict labels for test samples

STEP 6: Evaluate Performance
1. Compare predicted labels with actual labels
2. Compute Accuracy
3. Compute Precision, Recall, and F1-Score
4. Display Classification Report

STEP 7: Classify New Input
1. Accept new raw text input
2. Preprocess and vectorize using same vectorizer
3. Predict class label using trained model
4. Return predicted label

OUTPUT:
Predicted labels for test samples
Model accuracy and classification report
Predicted category for new input text
END

Result

```
                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive

Dataset size: (50000, 2)
```

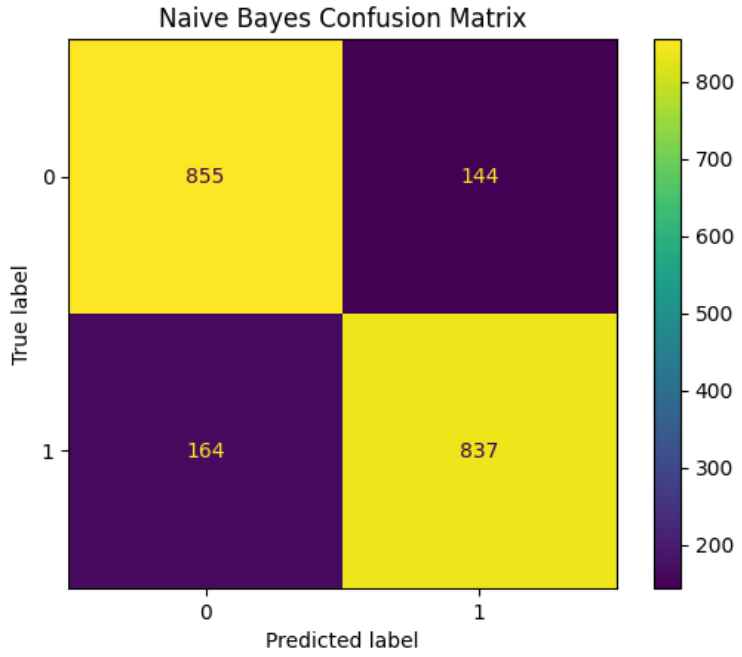
Fig 1 : dataset and sentiment

```
Naive Bayes Accuracy: 0.846
              precision    recall  f1-score   support

    negative       0.84      0.86      0.85       999
    positive       0.85      0.84      0.84      1001

    accuracy                           0.85      2000
   macro avg       0.85      0.85      0.85      2000
weighted avg       0.85      0.85      0.85      2000


SVM Accuracy: 0.866
              precision    recall  f1-score   support

    negative       0.89      0.84      0.86       999
    positive       0.85      0.89      0.87      1001

    accuracy                           0.87      2000
   macro avg       0.87      0.87      0.87      2000
weighted avg       0.87      0.87      0.87      2000
```

Fig 2 : model results

```
Input: this movie was absolutely amazing and fantastic
Naive Bayes: positive
SVM: positive

Input: worst movie ever i hated it completely
Naive Bayes: negative
SVM: negative
```

Fig 3 : Testing result



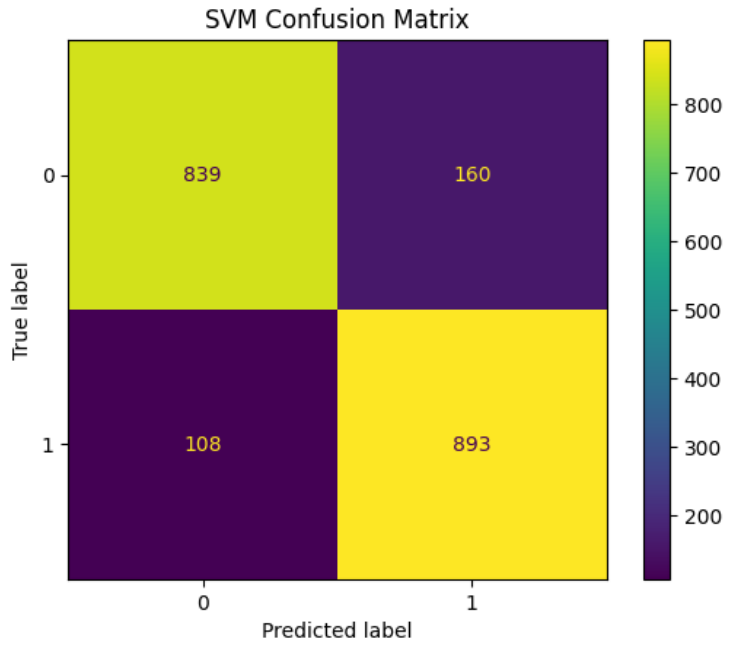


Fig 4 : confusion Matrix

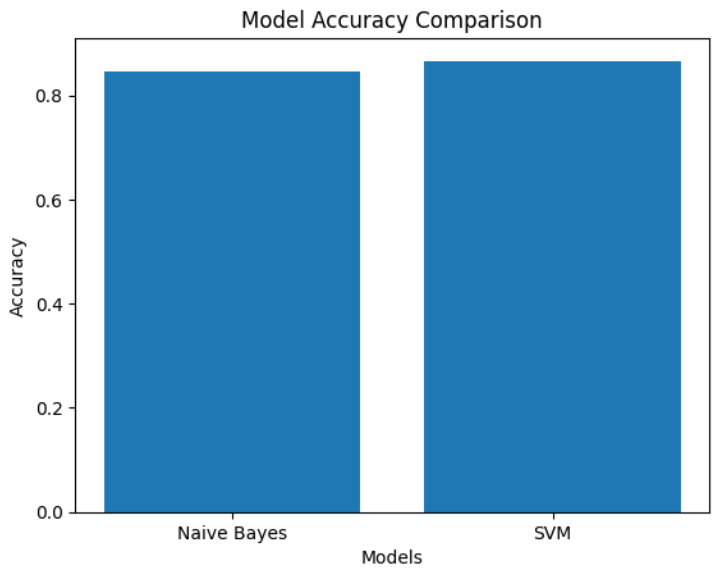


Fig 5 : comparison

Conclusion

The text classification model successfully classifies text samples into predefined categories using Naive Bayes or SVM. By applying text preprocessing and TF-IDF feature extraction, the model captures important word patterns and effectively distinguishes between classes. Both Naive Bayes and SVM prove to be efficient and accurate classifiers for NLP tasks, with SVM generally achieving higher accuracy on complex datasets. This experiment demonstrates the practical application of machine learning algorithms in natural language understanding and lays the foundation for more advanced text mining and classification systems.

Github Link: https://github.com/rachanadixit/NLP/blob/main/Ass6_NLP_138.ipynb